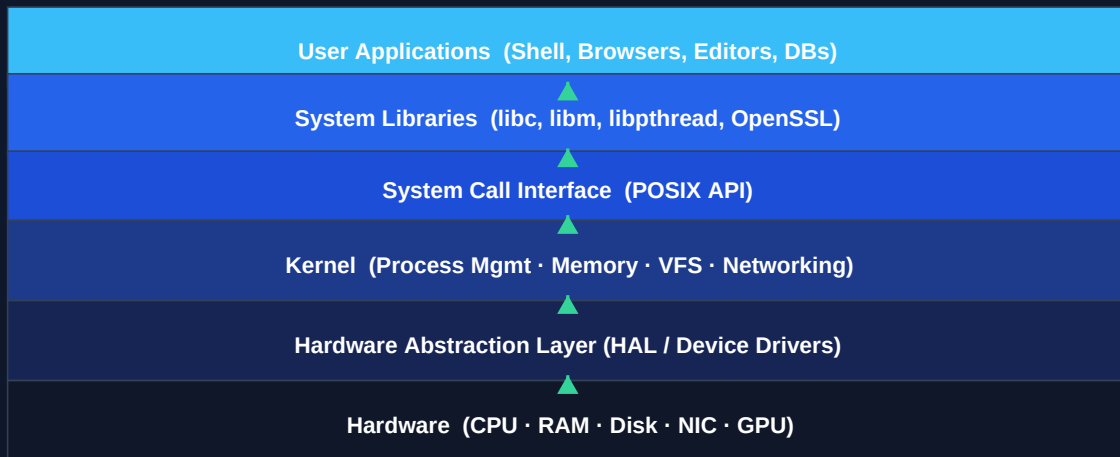


OS Architecture & Linux Fundamentals

Comprehensive Study Notes

Shell · Services · Utilities · Applications · Libraries · File System · APIs · System Calls



↑ User Space

↓ Kernel Space

Figure 1 — Complete OS Layered Architecture

Table of Contents

1. OS Architecture Overview

- 1.1 Monolithic vs Micro vs Hybrid Kernels
- 1.2 Kernel Subsystems
- 1.3 User vs Kernel Space

2. Linux Interfaces

- 2.1 CLI / Shell Interface
- 2.2 GUI (X11 & Wayland)
- 2.3 Socket / Network Interface
- 2.4 D-Bus & IPC

3. The Shell

- 3.1 Shell Variants
- 3.2 Execution Pipeline
- 3.3 Scripting Constructs
- 3.4 Job Control

4. Services & Daemons

- 4.1 systemd Architecture
- 4.2 Service Units
- 4.3 Service Lifecycle
- 4.4 Targets & Runlevels

5. Utilities & Core Tools

- 5.1 File Utilities
- 5.2 Process Utilities
- 5.3 Network Utilities
- 5.4 Text Processing

6. Applications

- 6.1 Application Types
- 6.2 Package Management
- 6.3 Application Packaging

7. Libraries

- 7.1 Static Libraries
- 7.2 Shared Libraries
- 7.3 Dynamic Linking
- 7.4 Construction of Libraries

8. File System

- 8.1 VFS Layer
- 8.2 FHS Directory Structure
- 8.3 File Types & Permissions
- 8.4 Common FS Types

9. APIs & System Calls

- 9.1 POSIX API
- 9.2 System Call Mechanism
- 9.3 Key System Calls

1. OS Architecture Overview

An Operating System (OS) is the fundamental software layer that manages computer hardware resources and provides a stable, abstract environment for applications. The architecture defines how responsibilities are divided across layers — from raw silicon to end-user programs.

1.1 Kernel Types

Monolithic	All kernel services (memory, scheduling, drivers, FS) run in a single address space. Fast IPC but less fault isolation. Example: Linux, traditional Unix.
Microkernel	Only core services (IPC, basic scheduling) run in kernel space; everything else in user-space servers. More stable, slower IPC. Example: MINIX, QNX, Hurd.
Hybrid	Combines both: core services in kernel, drivers/servers optionally in user space. Example: Windows NT, macOS (XNU = Mach microkernel + BSD).
Exokernel	Kernel exposes raw hardware to library OSES; maximum performance flexibility. Research systems (MIT Exokernel).
Unikernel	Single-purpose VM image; app + necessary OS components compiled together. Used in cloud for performance/security.

1.2 Kernel Subsystems

The Linux kernel is monolithic and contains these major subsystems:

Subsystem	Responsibility	Key Components
Process Management	Create/schedule/terminate processes & threads	CFS Scheduler, fork(), clone()
Memory Management	Virtual memory, paging, swapping, slab allocator	MMU, page tables, kmalloc, mmap
Virtual File System	Unified file abstraction over all FS types	VFS layer, dcache, inode cache
Network Stack	TCP/IP networking, socket layer, drivers	sk_buff, netfilter, socket API
IPC	Inter-process communication primitives	Pipes, signals, semaphores, shared mem
Device Drivers	Abstraction for character/block/network devices	Driver model, udev, /dev
Security	DAC, MAC, namespaces, capabilities	SELinux, AppArmor, seccomp

1.3 User Space vs Kernel Space

User Space	Ring 3 (x86). Unprivileged. Cannot directly access hardware. Processes run here. Memory-protected by MMU.
-------------------	---

Kernel Space	Ring 0. Full hardware access. Page fault handlers, interrupt handlers, kernel threads run here.
Context Switch	CPU saves registers, switches page table (CR3 on x86), transfers control. Cost: ~1-10 μ s.
Protection Rings	x86 defines Ring 0–3. Linux only uses Ring 0 (kernel) and Ring 3 (user). Rings 1–2 unused.

2. Linux Interfaces

Linux exposes multiple interfaces through which users and programs interact with the kernel. These interfaces range from text-based shells to graphical environments and programmatic APIs.

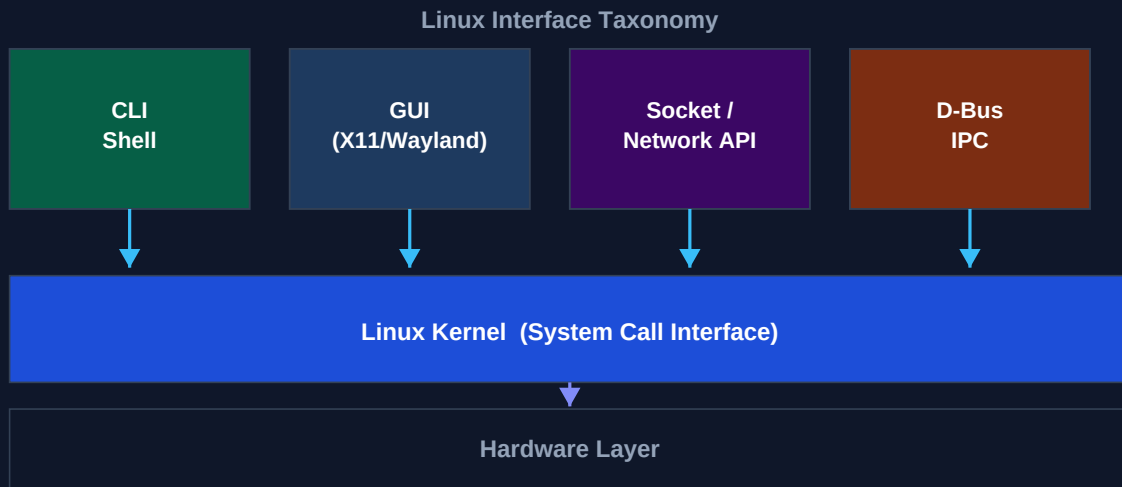


Figure 2 — Linux Interface Taxonomy

2.1 Command-Line Interface (CLI)

Terminal Emulator	gnome-terminal, xterm, alacritty — hosts the shell process and renders text
PTY Layer	Pseudo-terminal (pts/N) pairs master/slave; allows programs to talk to 'terminal' over file descriptors
Shell	bash, zsh, fish — reads input, interprets commands, manages processes
readline	Library providing command-line editing, history, auto-completion for interactive programs

2.2 Graphical Interface — X11 & Wayland

X Window System (X11) — client-server protocol; X server manages display hardware, clients (applications) send drawing commands over sockets. Over 40 years old; network-transparent but architecturally complex. **Wayland** — modern replacement; compositor speaks directly to kernel (DRM/KMS), less latency, better security. Most distros now default to Wayland (GNOME $\geq$ 42, KDE Plasma $\geq$ 6).

2.3 D-Bus & IPC Interfaces

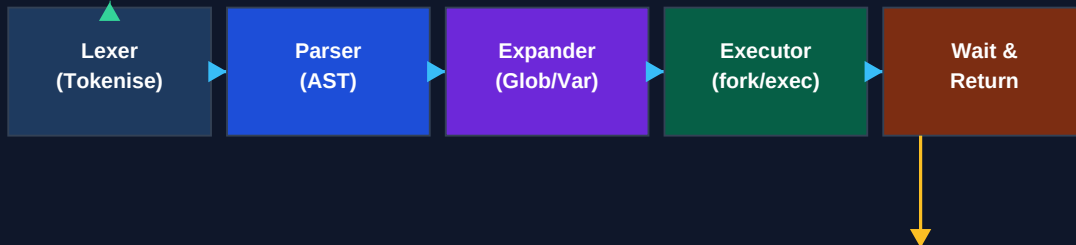
D-Bus	Message bus for IPC between desktop processes; used by systemd, NetworkManager, PulseAudio
Unix Sockets	SOCK_STREAM/DGRAM over file-system path; zero-copy option via SCM_RIGHTS

Named Pipes (FIFO)	mkfifo — byte-stream IPC between unrelated processes
Shared Memory	POSIX shm_open or SysV shmget — fastest IPC, no data copying
Signals	Asynchronous kernel→process notification (SIGTERM, SIGKILL, SIGSEGV ...)

3. The Shell

The shell is both a command interpreter and a high-level programming language. It bridges the user and the operating system by translating human-readable commands into system calls and process launches.

```
stdin: $ ls -la /home
```



stdout / stderr → Terminal / Pipe

Shell Command Execution Pipeline

Figure 3 — Shell Command Execution Pipeline

3.1 Shell Variants

Shell	Standard	Features	Config File
sh (Bourne)	POSIX	Minimal, universal scripting	/etc/profile
bash	POSIX++	Arrays, [], process sub, history	~/.bashrc ~/.bash_profile
zsh	bash compat	Better completion, themes (Oh My Zsh)	~/.zshrc
fish	Non-POSIX	Syntax highlighting, auto-suggest	~/.config/fish/config.fish
dash	POSIX	Fastest, used as /bin/sh on Debian/Ubuntu	/etc/profile
ksh	POSIX++	Origin of many bash features	~/.kshrc

3.2 Shell Scripting Constructs

Variables	name=value; echo \$name; readonly PI=3.14; export PATH
Arrays	arr=(a b c); echo \${arr[1]}; declare -A hash; hash[key]=val
Conditionals	if [cond]; then ... elif ...; else ...; fi [[extended]]
Loops	for x in list; do ... done while cond; do ... done until
Functions	myfunc() { local x=\$1; return 0; } -- local scoping important
Redirection	cmd > out 2>&1 cmd < in cmd 2>/dev/null cmd >> append
Pipelines	cmd1 cmd2 cmd3 -- stdout->stdin chaining; subshells per stage
Process Sub	diff <(cmd1) <(cmd2) -- feed command output as file descriptor

Here-doc	cat <<EOF ... EOF -- multi-line string literal to stdin
Arithmetic	\$((expr)) or let or bc for floats
Exit codes	\$? holds last exit code; 0 = success; set -e exits on error
Trap	trap 'cleanup' EXIT INT TERM -- run code on signals

3.3 Job Control

Foreground job	Command runs in foreground; shell waits. Ctrl+C sends SIGINT.
Background job	cmd & — shell does not wait; returns immediately with job ID.
jobs	List active background/stopped jobs with job IDs.
fg / bg	fg %1 brings job 1 to foreground; bg %1 resumes in background.
Ctrl+Z	Sends SIGTSTP to foreground process; suspends it (STOPPED state).
disown	Remove job from shell's job table; survives shell exit.
nohup	nohup cmd & — ignore SIGHUP, redirect output to nohup.out.

4. Services & Daemons (systemd)

A **daemon** is a long-running background process that provides system or network services. **systemd** is the dominant init system and service manager on modern Linux distros. It starts services in parallel, tracks dependencies, and manages the service lifecycle.

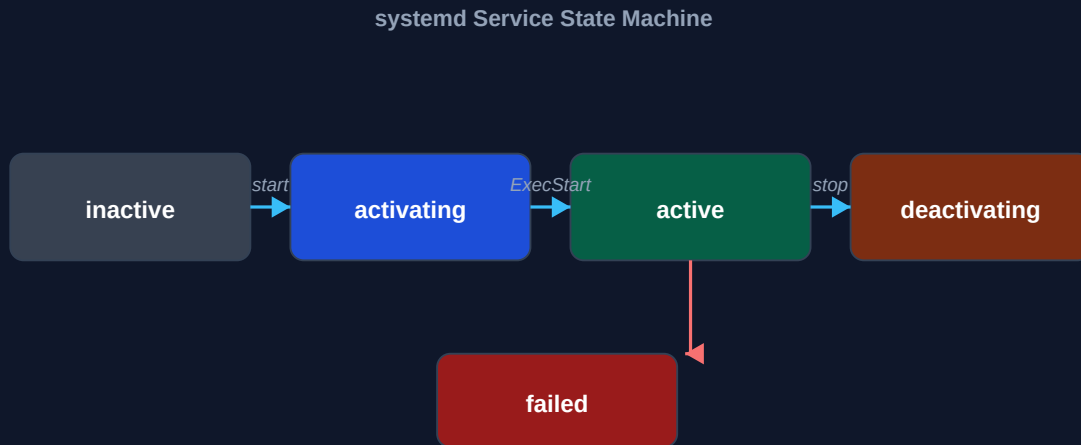


Figure 4 — systemd Service State Machine

4.1 systemd Unit File Structure

A `.service` unit file controls how systemd manages a service:

```
[Unit]
Description=My Application Server
After=network.target
Requires=postgresql.service

[Service]
Type=simple # simple | forking | oneshot | notify | dbus | idle
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server --port 8080
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure # always | on-failure | on-abnormal
RestartSec=5
StandardOutput=journal

[Install]
WantedBy=multi-user.target
```

4.2 Key systemctl Commands

systemctl start svc	Start a service immediately
systemctl stop svc	Stop a service immediately

systemctl restart svc	Stop + start
systemctl reload svc	Send SIGHUP (reload config without restart)
systemctl enable svc	Enable at boot (creates symlink in .wants)
systemctl disable svc	Disable at boot
systemctl status svc	Show status, last 10 log lines
systemctl list-units	List all loaded units
journalctl -u svc -f	Follow logs for a specific service
systemctl daemon-reload	Re-read unit files after modification

4.3 Service Types

simple	Default. ExecStart is the main process. systemd tracks its PID.
forking	Process daemonizes (double-fork). systemd follows child via PIDFile=.
oneshot	Short-lived; systemd waits for completion before next unit. RemainAfterExit=yes keeps 'active'.
notify	Process sends sd_notify(READY=1) when ready. Most reliable for complex apps.
dbus	Considered ready when specified BusName appears on D-Bus.
idle	Like simple but waits until all active jobs complete — prevents log interleaving.

5. Utilities & Core Tools

5.1 File Utilities

Command	Purpose	Common Options / Example
ls	List directory contents	-la (long+hidden), --color, -h (human sizes)
cp	Copy files/dirs	-r (recursive), -p (preserve attrs), -u (update)
mv	Move / rename	mv src dst — atomic if same filesystem
rm	Remove files/dirs	-rf (recursive force); CAUTION: no trash
find	Search by criteria	find / -name '*.log' -mtime +7 -delete
locate	Fast DB-backed search	locate nginx.conf; updatedb refreshes DB
stat	File metadata	inode, permissions, timestamps, block size
ln	Create links	-s for symlink; hard link shares inode
chmod	Change permissions	chmod 755 file; chmod u+x file
chown	Change owner/group	chown user:group file; -R recursive
du	Disk usage	du -sh * sorts by human size
df	Filesystem usage	df -h shows all mounted filesystems
tar	Archive tool	tar czf archive.tar.gz dir/ ; tar xzf to extract
rsync	Efficient sync	rsync -avz --delete src/ dst/

5.2 Process Utilities

Command	Purpose	Notes
ps	Process snapshot	ps aux — all processes; ps -ef; ps fax (tree)
top / htop	Live process monitor	htop: colour, mouse support, filters
kill / killall	Send signals	kill -9 PID (SIGKILL); killall nginx
nice / renice	CPU priority	nice -n 10 cmd; renice +5 -p PID
strace	Trace system calls	strace -p PID; -e trace=open,read
ltrace	Trace library calls	ltrace ./binary
lsof	List open files/sockets	lsof -i :8080; lsof -p PID
pmap	Process memory map	pmap -x PID
vmstat	Virtual memory stats	vmstat 1 10 (10 snapshots 1s apart)
iostat	I/O statistics	iostat -x 2 — extended disk stats
sar	System activity report	sar -u 1 5; sar -n DEV 1

5.3 Text Processing

grep	Search text with regex. <code>grep -rn 'pattern' dir/</code> <code>grep -v exclude</code>
sed	Stream editor. <code>sed 's/old/new/g' file</code> ; <code>sed -n '5,10p' file</code>
awk	Pattern-action processing. <code>awk '{print \$1,\$3}'</code> <code>awk -F: '{sum+=\$3} END{print sum}'</code>
sort	Sort lines. <code>sort -k2 -n -r</code> (field 2, numeric, reverse)
uniq	Filter duplicate lines. <code>sort file uniq -c</code> (count occurrences)
cut	Extract fields. <code>cut -d: -f1,3 /etc/passwd</code>
tr	Translate chars. <code>tr 'a-z' 'A-Z'</code> ; <code>tr -d '\n'</code>
wc	Word/line/byte count. <code>wc -l file</code> ; <code>find . wc -l</code>
head/tail	First/last N lines. <code>tail -f /var/log/syslog</code> (follow)
xargs	Build commands from stdin. <code>find . -name '*.tmp' xargs rm</code>
tee	Read stdin, write to file AND stdout. <code>cmd tee output.log</code>
jq	JSON processor. <code>curl api jq '.items[].name'</code>

5.4 Network Utilities

ip	Modern iproute2. <code>ip addr show</code> ; <code>ip route add</code> ; <code>ip link set eth0 up</code>
ss / netstat	Socket statistics. <code>ss -tulnp</code> (TCP/UDP listening with pids)
ping / traceroute	Connectivity and path tracing. <code>mtr</code> for continuous traceroute
curl / wget	HTTP requests. <code>curl -X POST -H 'Content-Type: application/json' -d '{}'</code>
nmap	Network scanner. <code>nmap -sV -p 1-1024 host</code>
tcpdump	Packet capture. <code>tcpdump -i eth0 port 80 -w capture.pcap</code>
nc (netcat)	TCP/UDP Swiss army knife. <code>nc -l 8080</code> ; <code>nc host 8080 < file</code>
dig / nslookup	DNS queries. <code>dig +short google.com MX</code>
iptables / nftables	Firewall rules. <code>iptables -A INPUT -p tcp --dport 22 -j ACCEPT</code>
ssh	Secure remote shell + tunnelling. <code>ssh -L 8080:internal:80 bastion</code>

6. Applications

6.1 Application Types on Linux

Native Binary	ELF executable; directly executed by the kernel via <code>execve()</code> . Platform-specific.
Interpreted Script	Python, Perl, Ruby scripts. Kernel runs interpreter via shebang (<code>#!</code>) line.
JVM / Managed	Java, Kotlin, Scala; JVM provides runtime, JIT compilation, GC on top of OS.
GUI Application	Uses toolkit (GTK, Qt) which talks to display server (X11/Wayland).
Daemon / Service	Background process managed by systemd; no terminal attached.
Container	Isolated via cgroups + namespaces; shares host kernel. Docker, Podman, LXC.
Flatpak / Snap	Self-contained with bundled runtime; sandbox via bubblewrap + portals.
Web Application	Browser-hosted; HTML/JS/CSS; may call backend APIs over HTTPS.

6.2 Package Management

Distro Family	Low-level	High-level	Package Format
Debian/Ubuntu	dpkg	apt / apt-get	.deb
RHEL/Fedora/CentOS	rpm	dnf / yum	.rpm
Arch	pacman	pacman / yay (AUR)	.pkg.tar.zst
SUSE	rpm	zypper	.rpm
Universal	—	Flatpak / Snap / AppImage	.flatpak / .snap / .AppImage

6.3 ELF Binary Format

Linux executables use the **Executable and Linkable Format (ELF)**. ELF files have a header, program headers (segments for runtime), and section headers (sections for linking). Key segments: `LOAD` (mapped into memory), `DYNAMIC` (dynamic linker info), `INTERP` (path to interpreter, e.g. `/lib64/ld-linux-x86-64.so.2`).

ELF Header	Magic bytes (0x7F ELF), architecture, entry point address, offsets to PHT/SHT
LOAD segment	Mapped by kernel; separate for <code>.text</code> (r-x) and <code>.data</code> (rw-)
.text section	Executable machine code

.data / .bss	.data: initialised globals; .bss: zero-initialised globals (no file space)
.rodata	Read-only data: string literals, constants
.plt / .got	Procedure Linkage Table / Global Offset Table for dynamic linking
.symtab / .strtab	Symbol and string tables (stripped in production binaries)
readelf / objdump	Inspect ELF: readelf -a binary; objdump -d binary (disassembly)

7. Libraries & Their Construction

Libraries are collections of pre-compiled code that applications link against to reuse functionality. Linux supports two main forms: **static libraries (.a)** compiled into the binary, and **shared/dynamic libraries (.so)** loaded at runtime.

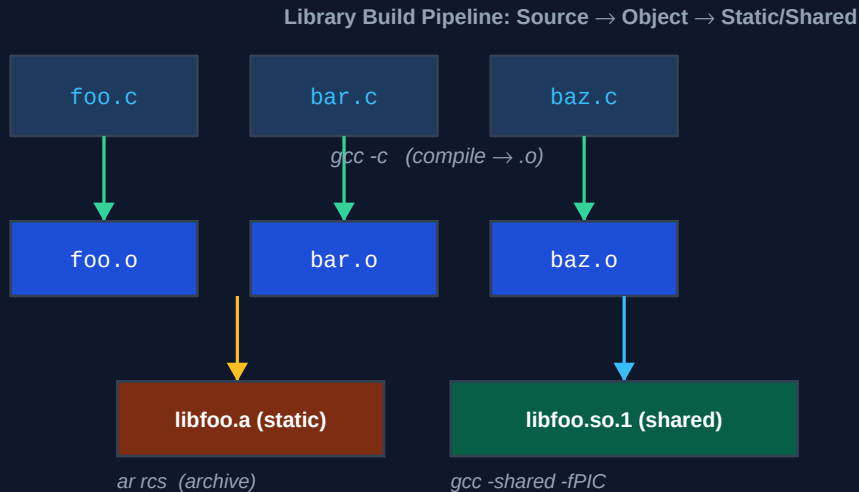


Figure 5 — Library Build Pipeline

7.1 Static Libraries (.a)

An archive of .o files. Linked at compile time — code copied into final binary.

```
# Compile source files
gcc -c foo.c -o foo.o
gcc -c bar.c -o bar.o
# Create archive
ar rcs libmylib.a foo.o bar.o
# Link application against static library
gcc main.c -L. -lmylib -o app_static
# Check: binary contains lib code
nm app_static | grep foo_function
```

7.2 Shared Libraries (.so)

Shared libraries are loaded into process address space by the **dynamic linker** (`ld-linux.so`) at startup (or lazily with `dlopen`). Multiple processes share one copy in physical memory — major RAM saving.

```
# Compile with Position-Independent Code (essential for .so)
gcc -c -fPIC foo.c -o foo.o
gcc -c -fPIC bar.c -o bar.o
# Create shared library with SONAME
gcc -shared -Wl,-soname,libmylib.so.1 -o libmylib.so.1.0.0 foo.o bar.o
# Create symlinks (ldconfig handles this automatically)
ln -s libmylib.so.1.0.0 libmylib.so.1
ln -s libmylib.so.1 libmylib.so
# Compile app against shared library
```

```
gcc main.c -L. -lmylib -o app_shared
# Update shared library cache
sudo ldconfig
# Check dependencies
ldd app_shared
```

7.3 Dynamic Linking & Loading

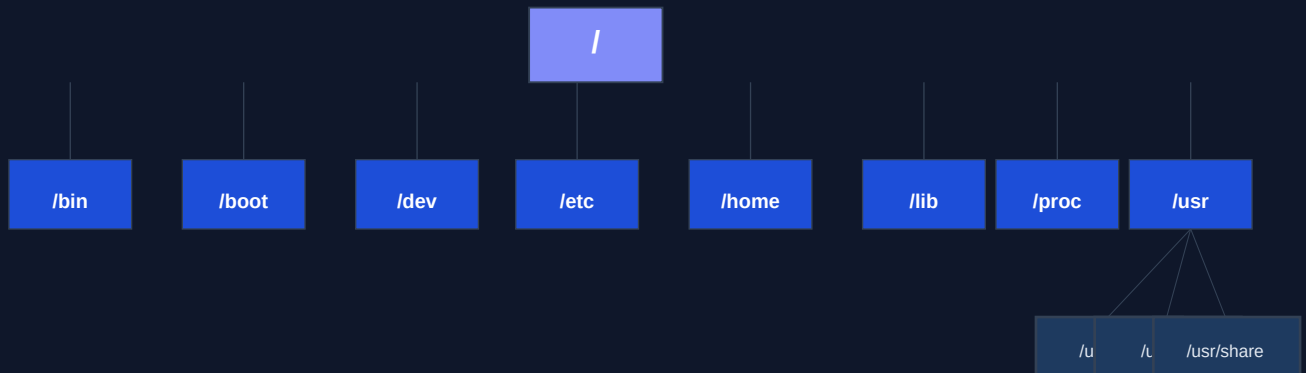
SONAME	Embedded library name (libfoo.so.1). Versioned; changing ABI increments major version.
ldconfig	Scans /etc/ld.so.conf dirs, creates /etc/ld.so.cache; run after installing .so files.
LD_LIBRARY_PATH	Env var overriding library search path. Useful for testing, bad for production.
LD_PRELOAD	Inject a library before all others. Used for sandboxing, malloc replacements.
dlopen / dlsym	Runtime dynamic loading: <code>void *h = dlopen('libfoo.so', RTLD_LAZY); dlsym(h, 'func')</code>
PLT / GOT	First call goes through PLT to resolver which patches GOT entry; subsequent calls direct.
RPATH / RUNPATH	Embed library search path in binary. rpath searched before LD_LIBRARY_PATH.

7.4 Important System Libraries

Library	Purpose	Key Symbols
glibc (libc.so.6)	C standard library; syscall wrappers, printf, malloc	printf, malloc, fopen, pthread_create
libm.so.6	Math functions	sin, cos, sqrt, pow, log
libpthread.so.0	POSIX threads	pthread_create, mutex, cond, rwlock
libdl.so.2	Dynamic loading	dlopen, dlsym, dlclose, dLError
libssl / libcrypto	TLS/SSL (OpenSSL)	SSL_CTX_new, EVP_*, BIO_*
libz.so.1	zlib compression	deflate, inflate, gzip
libcurl	URL transfer library	curl_easy_init, curl_easy_perform
libsystemd	systemd integration	sd_notify, sd_journal_print

8. File System

Linux abstracts all storage and I/O as files. The **Virtual File System (VFS)** provides a uniform API regardless of the underlying filesystem type (ext4, XFS, tmpfs, procfs ...). The **Filesystem Hierarchy Standard (FHS)** defines where files live.



Linux File System Hierarchy Standard (FHS)

Figure 6 — Linux FHS Directory Tree

8.1 FHS Key Directories

/	Root of the entire filesystem hierarchy
/bin	Essential user command binaries (ls, cp, mv, bash). Symlinked to /usr/bin on modern distros.
/boot	Kernel images (vmlinuz), initramfs, GRUB config
/dev	Device files. /dev/sda (disk), /dev/null, /dev/random, /dev/tty, /dev/mem
/etc	System-wide configuration files. /etc/fstab, /etc/passwd, /etc/ssh/sshd_config
/home	User home directories. /home/username/
/lib	Shared libraries needed by /bin and /sbin. Kernel modules in /lib/modules/
/media	Mount point for removable media (USB, CD)
/mnt	Temporary mount points for sysadmin use
/opt	Optional/third-party software packages (e.g., /opt/google)
/proc	Virtual FS exposing kernel/process info. /proc/cpuinfo, /proc/meminfo, /proc/PID/
/root	Home directory for the root user
/run	Runtime data since last boot (PID files, sockets). tmpfs — cleared on boot

/sbin	System binaries for root (fdisk, ip, iptables)
/sys	Virtual FS (sysfs) for kernel/hardware topology and configuration
/tmp	Temporary files; cleared on boot (often tmpfs)
/usr	User utilities and applications. Largest hierarchy. /usr/bin, /usr/lib, /usr/share
/var	Variable data: logs (/var/log), spool (/var/spool), databases, caches

8.2 File Types

- (regular)	Ordinary file: text, binary, script, archive
d (directory)	Container for other files and directories
l (symlink)	Symbolic link — pointer to another path
c (char device)	/dev/tty, /dev/null — byte-stream I/O
b (block device)	/dev/sda — random-access block I/O
p (named pipe)	FIFO for IPC between processes
s (socket)	Unix domain socket for IPC

8.3 Permissions & Ownership

Every file has an owner (user), owning group, and three permission sets (owner/group/others). Each set has read (r=4), write (w=2), execute (x=1) bits. Special bits: SUID (runs as file owner), SGID (runs as group / new files inherit group), Sticky (only owner can delete).

```
# ls -l output: -rwxr-xr-- 1 alice devs 4096 Jun 1 myfile
# ^--- type+perms owner group
chmod 755 file # rwxr-xr-x
chmod u+s /usr/bin/passwd # set SUID bit
chmod g+s /shared/dir # set SGID on directory
chmod +t /tmp # set sticky bit
chown alice:devs file
getfacl file / setfacl -m u:bob:rw file # ACLs (extended permissions)
```

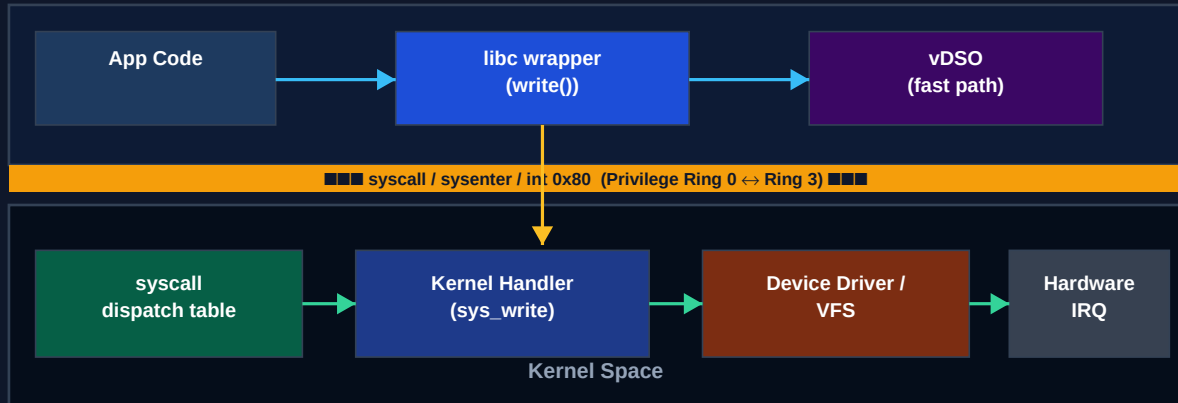
8.4 Common Filesystem Types

FS Type	Use Case	Max File Size	Max FS Size	Features
ext4	General Linux default	16 TB	1 EB	Journaling, extents, htree dirs
XFS	Large files/DBs	8 EB	8 EB	Journaling, parallel I/O, online grow
Btrfs	Modern, snapshotting	16 EB	16 EB	CoW, snapshots, subvolumes, RAID
tmpfs	RAM-backed temp storage	RAM limit	RAM limit	Lost on reboot; /tmp, /run
procfs	Kernel process info	N/A	N/A	Virtual; /proc/PID/*, /proc/sys
sysfs	Kernel hardware topology	N/A	N/A	Virtual; /sys/bus, /sys/class

FAT32	USB/SD interop	4 GB	2 TB	No permissions; universal compat
NTFS	Windows interop	16 TB	256 TB	ntfs-3g driver on Linux
NFS	Network file sharing	FS-dep	FS-dep	Remote mount over network
ZFS	Enterprise storage	16 EB	256 ZB	Pool-based, checksums, dedup

9. APIs & System Calls

An **API (Application Programming Interface)** defines a set of functions, protocols, and tools for building software. The OS API lets programs request OS services. In Linux, the primary API is **POSIX**; the underlying mechanism is the **system call**.



System Call Execution Flow (User → Kernel → Hardware)

Figure 7 — System Call Execution Flow

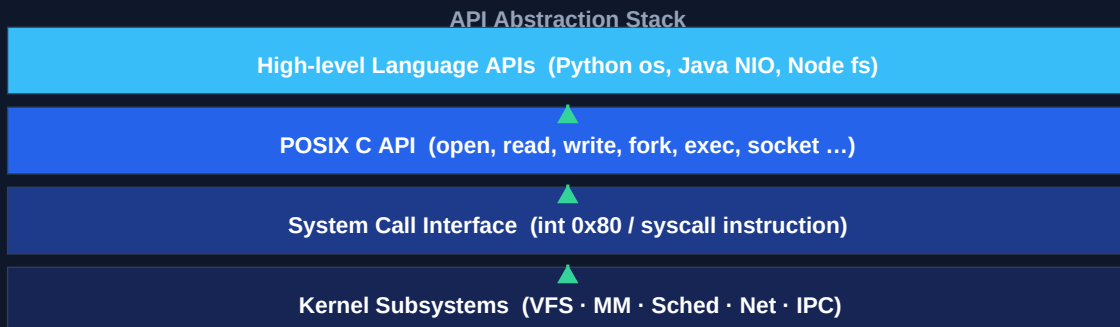


Figure 8 — API Abstraction Stack

9.1 POSIX API Categories

Category	Key Functions	Header
File I/O	open, close, read, write, lseek, fsync, mmap	<fcntl.h>, <unistd.h>
Directory	opendir, readdir, closedir, mkdir, rmdir, chdir	<dirent.h>
Process	fork, exec*, wait, waitpid, exit, getpid, getppid	<unistd.h>
Signals	signal, sigaction, kill, raise, sigsuspend, sigprocmask	<signal.h>
IPC	pipe, mkfifo, shm_open, sem_open, mq_open	<unistd.h>, <mqueue.h>
Threads	pthread_create, pthread_join, mutex, cond, rwlock	<pthread.h>
Sockets	socket, bind, listen, accept, connect, send, recv	<sys/socket.h>

Memory	malloc, free, mmap, munmap, brk, mprotect	<stdlib.h>, <sys/mman.h>
Time	time, gettimeofday, clock_gettime, nanosleep	<time.h>
User/Group	getuid, getgid, setuid, getpwnam, getgrnam	<unistd.h>, <pwd.h>

9.2 System Call Mechanism (x86-64)

Invocation	User program places syscall number in RAX, args in RDI,RSI,RDX,R10,R8,R9 registers; executes SYSCALL instruction.
Privilege Switch	CPU switches to Ring 0, saves user stack (SWAPGS, RSP), jumps to syscall entry point via MSR_LSTAR.
Dispatch	Kernel syscall handler indexes sys_call_table[RAX] to find the kernel function.
Execution	Kernel function runs (e.g. sys_read). Access user-space pointers via copy_from_user / copy_to_user.
Return	Result placed in RAX; SYSRET instruction returns to Ring 3. Negative values = -errno.
vDSO	Virtual Dynamic Shared Object — kernel maps read-only page into every process for fast syscalls (gettimeofday, clock_gettime) that don't need Ring 0 switch.

9.3 Essential System Calls Reference

Syscall	Number (x86-64)	Description
read	0	Read bytes from fd into buffer
write	1	Write bytes from buffer to fd
open	2	Open/create file, return fd
close	3	Close file descriptor
stat	4	Get file metadata via path
mmap	9	Map file/device into memory
mprotect	10	Set memory region protection
munmap	11	Unmap memory region
brk	12	Extend/shrink heap
ioctl	16	Device-specific control
pipe	22	Create anonymous pipe pair
dup2	33	Duplicate fd to specific number
fork	57	Create child process (copy-on-write)
execve	59	Replace process image
exit	60	Terminate process

wait4	61	Wait for child process
kill	62	Send signal to process/group
socket	41	Create socket endpoint
connect	42	Initiate connection on socket
accept	43	Accept incoming connection
bind	49	Bind socket to address
listen	50	Mark socket as passive
sendto	44	Send message on socket
recvfrom	45	Receive message from socket
clone	56	Create child with shared resources (threads)
futex	202	Fast user-space mutex (base of pthreads)
epoll_wait	232	Wait for I/O events (non-blocking I/O)
openat	257	Open relative to dir fd (modern open)
fanotify / inotify	253+	Filesystem event notification

9.4 Signals Reference

Signal	Number	Default Action	Meaning
SIGTERM	15	Terminate	Graceful termination request (catchable)
SIGKILL	9	Terminate	Unconditional kill — cannot be caught or ignored
SIGINT	2	Terminate	Keyboard interrupt (Ctrl+C)
SIGQUIT	3	Core dump	Keyboard quit (Ctrl+\) — generates core
SIGHUP	1	Terminate	Hangup — terminal closed; daemons use for reload
SIGSTOP	19	Stop	Pause process — cannot be caught or ignored
SIGCONT	18	Continue	Resume stopped process
SIGTSTP	20	Stop	Keyboard stop (Ctrl+Z) — catchable
SIGSEGV	11	Core dump	Segmentation fault — invalid memory access
SIGBUS	7	Core dump	Bus error — bad memory alignment / mapping
SIGFPE	8	Core dump	Floating-point exception
SIGPIPE	13	Terminate	Write to broken pipe — no reader
SIGALRM	14	Terminate	Alarm clock (set by alarm()/setitimer())
SIGUSR1	10	Terminate	User-defined signal 1
SIGUSR2	12	Terminate	User-defined signal 2
SIGCHLD	17	Ignore	Child stopped or terminated
SIGWINCH	28	Ignore	Terminal window size change

Quick Reference Summary

Topic	Key Points
OS Architecture	Hardware → HAL → Kernel → System Calls → Libraries → Applications. Kernel = Ring 0; User = Ring 3.
Linux Interfaces	CLI (Shell), GUI (X11/Wayland), Sockets, D-Bus, Proc/Sys virtual FS.
Shell	bash/zsh/fish. Pipeline: Lexer→Parser→Expander→fork/exec. Variables, loops, functions, redirection, job con
Services	systemd units (.service). States: inactive→activating→active→deactivating. journalctl for logs.
Utilities	File: ls/cp/mv/find. Process: ps/top/strace/lsof. Text: grep/sed/awk. Net: ip/ss/curl/tcpdump.
Applications	ELF binaries, scripts, JVM. Packages: apt/dnf/pacman. Containers via cgroups+namespaces.
Libraries	Static (.a, ar rcs), Shared (.so, -fPIC -shared). Dynamic linker: ld-linux.so. ldconfig, ldd, LD_LIBRARY_PATH.
File System	VFS abstraction. FHS: / /bin /etc /home /lib /proc /sys /var. Perms: rwxrwxrwx. ext4/XFS/Btrfs/tmpfs.
APIs & Syscalls	POSIX API (open/read/write/fork/exec/socket). Syscall: RAX=number, SYSCALL instruction, Ring 0, SYSRET